

CrediFlow — Technical Architecture & Stack Deep Dive

Deterministic GST Compliance, Dual-Agent Resolution & High-Throughput Analytics

System Overview: CrediFlow is an enterprise-grade GST Input Tax Credit (ITC) reconciliation and vendor resolution platform built under **Rule 60 CGST** and **Section 16(2)(aa)**. It combines a sub-millisecond deterministic reconciliation engine with an autonomous dual-agent dispute resolver, human-in-the-loop oversight, and automated bilingual recovery notice generation.

1. Technology Stack Summary Matrix

Layer	Technologies & Libraries	Version / Engine	Core Functionality & Role
Frontend UI	React, Vite, Tailwind CSS, Lucide React, Canvas Confetti	React 19.2 Vite 8.2	Single-page reactive dashboard, live audit ledger, HITL modal, analytics charts.
Backend API	FastAPI, Uvicorn, Pydantic v2, HTTPX, Python-Multipart	Python 3.10+ FastAPI 0.115+	Asynchronous REST microservices, file parsing, validation, batch simulation endpoints.
Persistence	SQLite 3, In-Memory State Store, Python Dataclasses	SQLite 3 In-Memory Cache	Persists audit ledger, benchmarks, human decisions, and dispatches; instant live cache.
Data Pipes	Tinybird / ClickHouse SQL Pipes (.pipe format)	ClickHouse SQL	High-throughput streaming ingestion, real-time aggregation, mismatch detection pipelines.
AI & Multi-Agent	RocketRide Dual-Agent Pipeline (OpenAI backend), Statutory Engine	Dual-Agent Statutory Fallback	Agent A (Root-Cause Classifier) + Agent B (Audit Cross-Examiner) + Deterministic Math.
Doc Engine	ReportLab PDF Toolkit, Bilingual Template Engine	ReportLab 5.0.1	Dynamic generation of legal GST Rule 60 recovery notices and bilingual Hindi/English nudges.
Deployment	Vercel Serverless, Static Assets Build, ASGI Adapter	Vercel / POSIX	Cloud serverless deployment with fallback disk paths and live production routing.

2. Frontend & Backend Architectural Stack

Frontend Ecosystem (React 19 + Vite 8 + Tailwind CSS)

- **React 19 (v19.2.8):** Utilizes React 19 concurrent rendering for fluid interactions across heavy multi-thousand invoice tables.
- **Vite 8 (v8.2.2):** High-speed bundler with Hot Module Replacement (HMR) and optimized ES module compilation.
- **Tailwind CSS & Utilities:** Styled with clsx (v2.1.1) and tailwind-merge (v3.6.0) for dark/light themes and glassmorphism styling.
- **Lucide React (v1.34.0):** Clean icon set for financial states, status badges, and AI agent flows.
- **Oxlint (v1.79.0):** Rust-based high-speed linter for strict code hygiene.
- **Canvas Confetti (v1.9.4):** Micro-interactions celebrating resolved reconciliation mismatches.

Backend Framework & API Layer (Python + FastAPI)

- **FastAPI (v0.115.0+):** Asynchronous REST API framework built on Starlette and OpenAPI specs for ultra-fast non-blocking execution.
- **Pydantic v2:** High-speed Rust-core serialization and schema validation for invoices, GSTINs, and audit decisions.
- **HTTPX (v0.27.0):** Asynchronous HTTP client powering external webhook calls to the RocketRide AI cloud.
- **OpenPyXL & Python-Multipart:** Multi-format data parser handling Excel (.xlsx), CSV, and JSON data uploads.
- **Uvicorn:** ASGI production server handling high-concurrency requests.

3. Data Layer, Storage & Streaming Pipelines

Persistence Architecture (SQLite + In-Memory + ClickHouse Pipes)

Component	Storage Type	Description & Table Schemas
audit_ledger	SQLite Table	Stores historical invoice audit records, supplier GSTINs, taxable amounts, tax breakdowns (CGST/SGST/IGST), mismatch flags, and statutory verdicts.
benchmark_runs	SQLite Table	Persists performance benchmarks: invoice counts (100 to 10,000+), runtime in ms, throughput (invoices/sec), total exposure INR, AI cost USD/INR, and retries.
human_decisions	SQLite Table	Audit trail of human interventions: gate IDs, invoice numbers, decisions (APPROVED/EDITED/REJECTED), reviewer identity, notes, and timestamps.
notice_dispatches	SQLite Table	Maintains dispatch logs for formal PDF notices: recipient supplier, delivery channel (WhatsApp/Email), PDF path, and dispatch status.
Session State	In-Memory Store	FastAPI in-memory singleton dictionary providing sub-millisecond query access for active frontend session reconciliation.
Streaming Pipes	ClickHouse / Tinybird (.pipe)	SQL analytics pipelines defined in pipelines/crediflow_audit_pipeline.pipe for streaming reconciliation, aggregate risk metrics, and batch ingestion.

4. Multi-Agent AI & Compliance Engine

- **Dual-Agent Architecture:**
 - **Agent A (Root-Cause Classifier):** Ingests multi-table GST discrepancies (Tax Mismatches, Missing Invoices, B2B classification errors, GSTR-1 vs 3B default) and determines root legal causation.
 - **Agent B (Audit Cross-Examiner):** Adversarially challenges Agent A's classification against statutory exemptions, Rule 60 compliance deadlines, and financial materialities.
- **Tri-Tier Execution Priority:**
 1. *RocketRide Webhook:* HTTPS POST to cloud multi-agent orchestration pipeline with OpenAI backend synthesis.
 2. *RocketRide SDK:* Direct WebSocket streaming (wss://) for cloud or edge execution.
 3. *Statutory Deterministic Fallback:* Pure Python compliance engine ensuring 100% mathematical precision (zero LLM calculation hallucinations) and 100% offline uptime.
- **Human-in-the-Loop (HITL) Gate:** Automatically routes high-exposure claims (> INR 50,000) or low-confidence verdicts (< 85%) to finance managers for review, modification, or rejection.

5. Document Generation & Communication Engine

- **ReportLab 5.0.1:** Programmatic generation of court-ready legal demand notices under GST Section 16(2)(aa) with custom tables, tax breakdowns, statutory citations, and formal letterheads.
- **Bilingual Nudge Engine:** Automated generation of dual-language communications (English & Hindi) customized with supplier risk scores and progressive escalation tones (Friendly Reminder → Urgent Statutory Warning).
- **Multi-Channel Dispatcher:** Dispatch simulation and connectivity for WhatsApp Business API and Email SMTP protocols.

6. Deployment, Packaging & Quality Assurance

- **Vercel Serverless Architecture:** Configured via vercel.json with Python serverless functions (api/index.py) and statically built React assets.
- **Modern Packaging:** Standardized with pyproject.toml (PEP 621) and npm modular package management.
- **Automated Test Suite:** Pytest suite (backend/tests/test_buildathon_requirements.py) validating Rule 60 compliance, reconciliation edge-cases, high-throughput scaling, and HITL gate logic.